

M1.3 — Bucket 1 claims

Live working companion to `docs/experiments/qualitative/M1.3-bucket1-claims.md`. Claims are refined one at a time; none are locked until their write-up is complete.

1 Preamble: notions of success

Throughout the Bucket 1 claims, four notions of success are kept distinct. They are not interchangeable, and no claim treats runner completion as mechanism recovery.

Procedural solved. The grid runner labels a cell `solved` when ABA Learning terminates and writes a solution artefact (`bk.sol.*`). That label records completion of the learning procedure. It does not entail full ASP coverage of $\langle E^+, E^- \rangle$, exact recovery of $\mathcal{H}_t^*$, or semantic success under the locked M1.2 description.

ASP coverage. After learning, positives covered and negatives rejected are checked under brave entailment on `bk.sol.asp`. Full coverage means the learned framework separates the example sets. Incomplete coverage (in particular, failure to reject negatives) may coexist with runner `solved`, as in ECAI U7 on x_3 .

Exact match to $\mathcal{H}_t^*$. Exact syntactic agreement with the locked reference rule set is recorded when it occurs. It is not the primary acceptance criterion for M1.3. Most cells that are scientifically interesting under the semantic success description are not exact matches.

Semantic success. Acceptance remains inspection-first against the locked M1.2 semantic success description for the cell. That description concerns whether the learned framework encodes the intended mechanism acceptably in the `val` language (including, where relevant, assumption-mediated presentations). Descendant citation falls outside semantic success. Exact match to $\mathcal{H}_t^*$ is neither necessary nor sufficient for semantic success.

Claims below use these four labels exactly as stated. Where a cell is runner-`solved` but fails ASP coverage, exact match, or semantic success, that failure is named explicitly.

2 Claim 1 — What AAMAS learned across M12x

Claim. Across all nine AAMAS M12x cells, under `folding_mode(greedy)`, `folding_selection(mgr)`, and `folding_space(bk)`, each positive row i is specialised to a target rule whose body cites the exact-value predicate of every non-target variable true on that row. Consequently $\text{Pa}(t) \subseteq \text{Vars}(B_r)$ for every learned target rule r , and no assumptions are introduced. Exact recovery of $\mathcal{H}_t^*$ occurs only on U2, where every available predictor is a parent and the locked reference already has the same positive-cell conjunction form.

Argument. Under greedy folding the engine walks each foldable body literal and accumulates every successful replacement by an eligible background head. With `folding_space(bk)`, those heads are drawn only from the original exact-value background. There is no later literal-pruning

or rule-compression stage. On the locked M12x tables, the resulting row-wise conjunctions already separate $\langle E^+, E^- \rangle$, so **gen2** accepts them without Assumption Introduction.

The Stage-3 inspections of cells 10–18 confirm the consequences. Parent variables appear in every AAMAS target body. In seven of the nine cells the bodies also retain a non-parent (isolated variable, sibling, ancestor, or descendant). On U3 the bodies cite only parents, yet the learned theory is an eight-clause positive-cell DNF rather than the compact unary OR of $\mathcal{H}_{x_2}^*$. Exact match occurs solely on U2 (cell 11), where the four both-parent conjunctions coincide with $\mathcal{H}_{x_2}^*$. That coincidence is representational: greedy’s natural output already matches the chosen reference form, and there are no distractors to retain. It is not evidence that AAMAS identified the parents as causally privileged.

3 Claim 2 — What ECAI learned across M12x

Claim. Across all nine ECAI M12x cells, under `folding_mode(nd)` with the token budget beginning at 1, learning commits early to one-literal folds on x_0 . When such a fold already separates $\langle E^+, E^- \rangle$, the engine keeps the corresponding unary target rule without assumptions. When it overgeneralises, Assumption Introduction under `asm_intro(relto)` patches that same x_0 -keyed body with an assumption. Contrary learning may then introduce further conditions, including conditions on other variables, but it does not replace the ordinary target body by a different backbone. As a result, every ECAI solution has ordinary target rules keyed by x_0 . No cell produces an ordinary multi-parent conjunction in the target rules.

That shared pattern separates the nine cells by the fixture fact $x_0 \in \text{Pa}(t)$. Where x_0 is a direct parent of the target—on U2, U3, both U4 targets, and U6 with target x_2 —the x_0 -keyed solutions are mechanism-aligned under the locked semantic success descriptions. Two of those cells are exact matches to $\mathcal{H}_t^*$. Where x_0 is not a direct parent—on U1, U5, U6 with target x_3 , and U7—the same pattern keys the target rules by a non-parent. Those cells are intensionally misaligned, even when a preferred parent later appears in a contrary, and even when ASP coverage is complete.

Argument. The ECAI configuration uses non-deterministic folding with an initial token budget of 1. At that budget a successful fold replaces the sample identifier by a single background literal. On the locked M12x background knowledge, the Stage-3 traces show that this first success is an exact-value literal of x_0 . Learning therefore begins from unary x_0 -keyed candidate target rules.

ABA Learning then treats overgeneralisation by Assumption Introduction relative to the failing body. The body is not discarded. An assumption α is added, giving a defeasible rule of the form $t(A) \leftarrow x_0_val_v(A), \alpha(A)$. Contrary learning may afterwards restrict when α holds, and those contrary conditions may mention other variables. In the ABA framework those contraries undercut the default. They do not substitute a new ordinary target backbone for the x_0 condition already chosen.

Two consequences follow for every ECAI cell. First, the ordinary target rules remain keyed by x_0 . Second, because repair proceeds around that one-literal backbone, the learner never emits an ordinary conjunction of several parent literals in the target rules. The Stage-3 inspections of cells 1–9 confirm both consequences.

The nine cells then divide on the role of x_0 in the locked graph. On U4, $\text{Pa}(t) = \{x_0\}$ for both targets. The unary x_0 folds already separate the examples, so the solutions are exact matches to $\mathcal{H}_t^*$ without assumptions. On U2, U3, and U6 x_2 , one likewise has $x_0 \in \text{Pa}(t)$. The early folds overgeneralise within their x_0 slices, and assumptions with contraries complete the encoding. In those cells a second parent may appear in a contrary as part of a coherent mechanism-aligned framework. The solutions succeed because the default backbone is already a direct parent.

On U1, x_0 is isolated. On U5, U6 x_3 , and U7, x_0 is a non-parent ancestor. The same procedure therefore builds a non-parent backbone. A preferred parent may still appear among the contrary conditions. That does not yield the intended parent-based target intension once the ordinary rules are keyed by a non-parent. On U6 x_3 and U7 the second parent x_2 is absent from the learned framework. U7 also fails to reject negatives under ASP coverage. The unsuccessful cells are the same ECAI pattern applied where $x_0 \notin \text{Pa}(t)$.

What ECAI does on M12x is therefore uniform: it learns x_0 -keyed target rules and uses assumptions and contraries only as repair around that backbone. What separates the cells is whether that backbone variable is a direct parent of t .

4 Claim 3 — Strong intensional ECAI solutions on U2, U3, and U6 x_2

Claim. On ECAI cells U2, U3, and U6 with target x_2 , the learned frameworks are theoretically defensible intensional solutions to the locked learning problems, not merely approximate coverage repairs. In each case the ordinary target rules, assumptions, and contrary-derived undercutting conditions jointly express the intended Boolean mechanism over the direct parents. The solutions achieve full ASP coverage. They cite neither sample identifiers nor non-parent predictors. On U6 x_2 they also leave the descendant unused. None of the three is an exact match to $\mathcal{H}_t^*$. The mismatch is therefore a difference of intensional presentation within ABA, not a failure to recover the target concept.

Argument. ABA Learning returns a complete assumption-based framework. Assumption Introduction and contrary learning are licensed transformations in that setting. Their purpose is to encode defeasible rules through undercutting. A solution that uses assumptions is therefore not, by that fact alone, an inferior substitute for an assumption-free rule set. What matters is whether the resulting framework expresses the intended mechanism.

Claim 2 already places these three cells among the ECAI successes with $x_0 \in \text{Pa}(t)$. The present claim concerns the quality of those particular assumption-mediated outputs. The Stage-3 inspections support a stronger reading than coverage alone.

On U2 the locked reference enumerates both-parents-nonzero as four ordinary parent conjunctions. The learned framework instead keeps unary nonzero conditions on x_0 in the target rules and expresses nonzero x_1 through undefeated assumptions whose contraries fire when $x_1 = 0$. On an $x_0 \in \{1, 2\}$ row, x_2 holds if and only if the assumption is undefeated if and only if $x_1 \neq 0$. That is exactly the nonzero-positive semantics of min.

On U6 with target x_2 the mechanism and the recovered intension are the same. The additional background predicates for the descendant x_3 are never cited. The framework again characterises both-parents-nonzero by an x_0 -keyed default together with contrary conditions on x_1 .

On U3 the locked reference is a four-clause expansion of at-least-one-parent-nonzero. The learned framework is an x_0 -first case analysis. The complete nonzero x_0 slices become ordinary unary rules. Only the incomplete $x_0 = 0$ slice needs an assumption, which encodes $x_1 \neq 0$ through its contrary. Extensionally this is $(x_0 = 0 \wedge x_1 \neq 0) \vee (x_0 = 1) \vee (x_0 = 2)$, which is exactly $x_0 \neq 0$ or $x_1 \neq 0$, the nonzero-positive semantics of max.

In all three cells the examples are fully separated under brave ASP coverage. The frameworks are variable-based rather than rote sample listings. They remain within the parent set of x_2 . Their departure from $\mathcal{H}_t^*$ is a choice of ABA presentation of the same Boolean concept. It should not be read as semantic failure.

5 Claim 4 — Shared incompleteness on U6 and U7 x_3 , exposed only on U7

Claim. ECAI cells U6 and U7 with target x_3 share the same parent set $\text{Pa}(x_3) = \{x_1, x_2\}$ and, on their respective observed tables, the same nonzero labelling rule $x_3 \neq 0$ if and only if $x_1 \neq x_2$. Under ECAI they also share the same structurally incomplete learned pattern. Ordinary target rules are keyed by the non-parent ancestor x_0 . The parent x_1 appears only in contrary conditions. The parent x_2 is absent from the learned framework. On U6 that incomplete framework fully separates $\langle E^+, E^- \rangle$. On U7 it covers all positives but rejects none of the three negatives. The coverage divergence is explained by support. On U6 the pair (x_0, x_1) already determines the x_3 labelling. On U7 it does not.

Argument. Both cells fall under Claim 2’s unsuccessful branch: $x_0 \notin \text{Pa}(x_3)$, so ECAI’s x_0 -first folding builds a non-parent backbone. The Stage-3 inspections show the same further incompleteness in both. After Assumption Introduction, contrary repair reaches conditions on x_1 . It never introduces an x_2 literal. Preferred parent x_2 remains unused on the accepted path. Relative to the locked two-parent references $\mathcal{H}_{x_3}^*$, both solutions are therefore structurally incomplete in the same way.

The mechanisms make the comparison controlled. On U6 one has $x_3 := x_1 - x_2$ with $x_2 := \min(x_0, x_1)$. On U7 one has $x_3 := |x_1 - x_2|$. In both cases the observed nonzero rule is $x_3 \neq 0$ if and only if $x_1 \neq x_2$, and the intended parents are $\{x_1, x_2\}$.

On U6 the table is the factorial over (x_0, x_1) . Because $x_2 = \min(x_0, x_1)$, one has $x_3 = 0$ whenever $x_1 \leq x_0$ and $x_3 > 0$ whenever $x_1 > x_0$. Under nonzero-positive labelling, x_3 is therefore positive exactly when $x_1 > x_0$, and negative otherwise. The pair (x_0, x_1) has deterministic case-based power over the target labelling. An x_0 -keyed default repaired by contrary conditions on x_1 can express that comparison without citing x_2 . The learned U6 theory does so, and ASP coverage is complete. Full separation therefore does not show that the omitted parent was unnecessary to the intended mechanism. It shows that $\{x_0, x_1\}$ already determined the labels on this support.

On U7 that determinacy fails. Each positive has a twin negative that shares (x_0, x_1) and differs only in x_2 . The pair (x_0, x_1) does not determine positivity for x_3 . Any framework that ignores x_2 must treat some twin pair alike. The same incomplete pattern therefore covers all positives and rejects none of the three negatives, despite runner **solved**.

U6 and U7 thus exhibit the same incomplete ECAI intension. U6’s coverage is an artefact of (x_0, x_1) determining the labels via $x_1 > x_0$. U7’s covering failure is that incompleteness made visible once (x_0, x_1) no longer determines the labels.